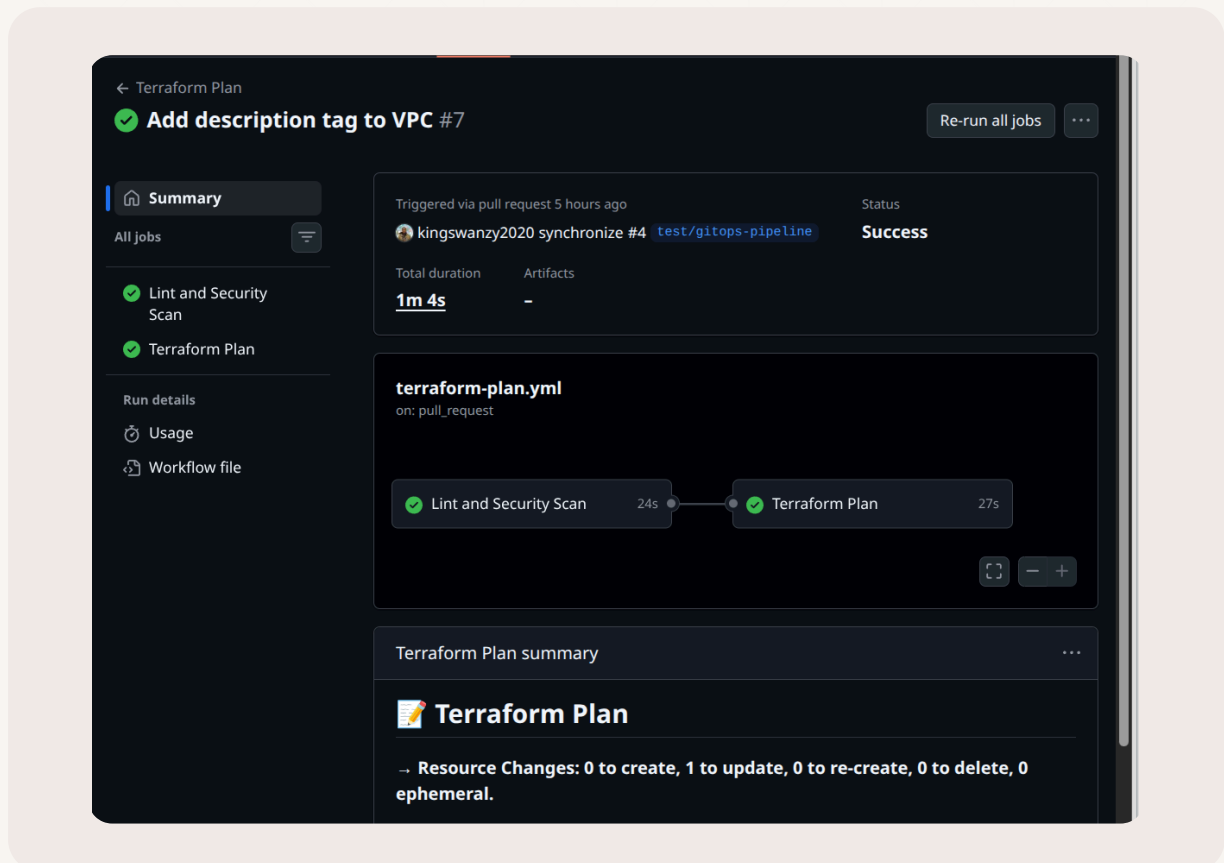


ENTERPRISE TERRAFORM GITOPS PIPELINE

By Ahmed Tetteh | May 2026



Project Vision: GitOps for Real-World Infrastructure

Why this pipeline matters

In this project, I'm building an Infrastructure as Code pipeline that mirrors how engineering teams manage and scale cloud infrastructure today. So infrastructure changes are versioned, controlled, and automatically applied upon a merge to the main branch.

Deploying a Live AWS Stack Through Code

Composing modules into a deployable environment

In this step, I'm creating root module and pass in all the right variables that connects all the other child modules together.

```
Apply complete! Resources: 34 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
instance_id = "i-064d65a9c26f653d0"  
instance_private_ip = "10.0.10.202"  
s3_bucket_id = "terraform-gitops-dev-202605140514127156000000004"  
vpc_id = "vpc-0327bela2b517d112"
```

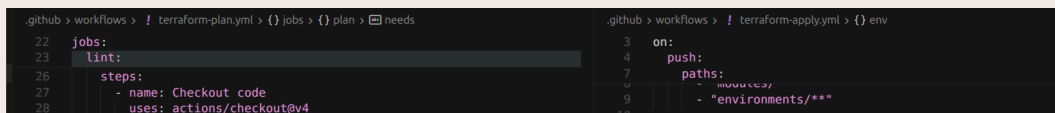
Remote state and locking for team safety

The S3 backend stores the terraform state file in an S3 bucket and DynamoDB prevents another process or entity from writing to the state file by creating a lock entry to the table.

Building the Full CI/CD Pipeline

Automating validation and deployment

In this step, I'm creating a full GitHub pipeline that tells GitHub Actions what to do when changes arrive, so that validations can be performed, automatic deployments done, and run a full GitOps cycle with real infrastructure changes.



```
.github > workflows > ! terraform-plan.yml > {} jobs > {} plan > needs
22 jobs:
23   lint:
26     steps:
27       - name: Checkout code
28         uses: actions/checkout@v4

.github > workflows > ! terraform-apply.yml > {} env
3 on:
4   push:
7     paths:
8       - modules/*
9       - "environments/**"
```

Plan vs. apply workflow triggers

The plan workflow triggers on a pull request, while the apply workflow triggers on a push to the main branch upon accepting a merge request. This matters because we need to review the infrastructure changes and catch coding errors before triggerings changes to the live infrastructure.

The complete GitOps cycle in action

I opened a pull request which triggered the GitHub actions workflow for the terraform plan. Then when I merged, it triggered the terraform apply workflow to automatically deploy the infrastructural changes. The result was immediate and seamless, as everything was now fully automated.

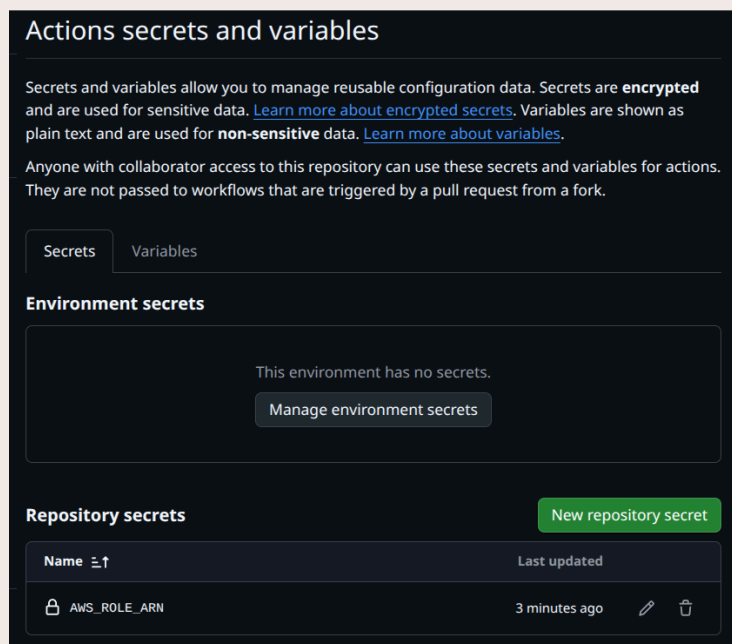
Passwordless AWS Authentication with GitHub OIDC

Setting up the OIDC identity provider

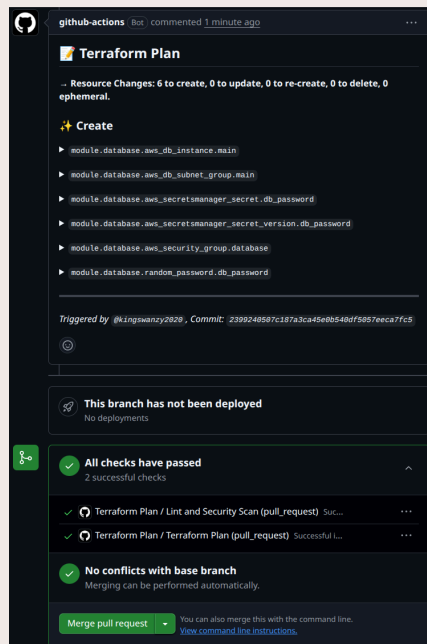
In this step, I'm setting up GitHub actions with OpenID Connect(OIDC) so that I can my AWS credentials are not stored permanently on GitHub as secrets.

OIDC vs. static credentials

The OIDC provider allows GitHub Actions to retrieve OIDC tokens and execute a workflow run on AWS, unlike static keys which requires some form of management and rotation to prevent leakage.



Secret Mission: Adding an RDS Database Module



Least-privilege database security group design

In this project extension, I configured the database security group to allow access only by the security group of the EC2 instance.

Bootstrapping Remote State and Project Structure

Installing tools and initializing the repository

In this step, I'm setting up the Terraform backend so that I can store the Terraform state file remotely, and installing some package requirements for the project.

```
.  
./environments  
./environments/dev  
./git  
./git/branches  
./git/hooks  
./github  
./github/workflows  
./git/info  
./git/logs  
./git/logs/refs  
./git/logs/refs/heads  
./git/objects  
./git/objects/a0  
./git/objects/bc  
./git/objects/c7  
./git/objects/info  
./git/objects/pack  
./git/refs  
./git/refs/heads  
./git/refs/tags  
./modules  
./modules/compute  
./modules/iam  
./modules/networking  
./modules/storage  
./venv  
./venv/bin
```

Solving the Terraform bootstrap problem

The bootstrap problem is a situation that implies that Terraform cannot manage an infrastructure containing its own state file. This also means that the Terraform state must exist before Terraform can be initialized for the project. I solved it by creating all the needed resources to store and perform the state lock outside of Terraform's management using the AWS CLI.

Building the Networking Module

VPC, subnets, and routing across availability zones

In this step, I'm setting up the Networking module for my resource deployment.

```
Terraform has created a lock file .terraform.lock.hcl to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.
```

```
Terraform has been successfully initialized!
```

```
You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.
```

```
If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.
```

```
Success! The configuration is valid.
```

Dynamic availability zone discovery

The data source is used because it dynamically fetches availability zones that are active in your region, which means the module can work in any AWS region without hardcoding.

Building Compute, Storage, and IAM Modules

EC2, S3, and least-privilege IAM

In this step, I'm building the compute, storage and IAM modules, so that the infrastructure has all the needed resources for the project.

```
• (.venv) king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps/terraform-gitops$ tree modules/  
modules/  
├── compute  
│   ├── main.tf  
│   ├── outputs.tf  
│   └── variables.tf  
├── iam  
├── networking  
│   ├── main.tf  
│   ├── outputs.tf  
│   └── variables.tf  
└── storage  
    ├── main.tf  
    ├── outputs.tf  
    └── variables.tf
```

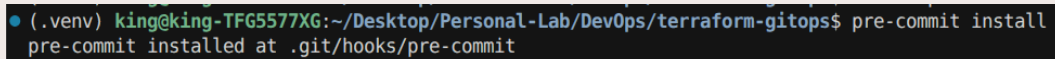
Security hardening applied to EC2

The compute module applies the `http_token=required`, which protects against credential theft attacks and encrypts the root volume.

Enforcing Code Quality with Pre-commit Hooks

Automating checks before every commit

In this step, I'm setting up pre-commit hooks that run automated checks upon every commit, so that I can catch any errors before they are deployed.

A terminal window with a dark background and light-colored text. The prompt is a blue dot followed by the text "(.venv) king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps/terraform-gitops\$". The command "pre-commit install" is entered. The output is "pre-commit installed at .git/hooks/pre-commit".

```
(.venv) king@king-TFG5577XG:~/Desktop/Personal-Lab/DevOps/terraform-gitops$ pre-commit install
pre-commit installed at .git/hooks/pre-commit
```

What the hooks catch and why it matters

My pre-commit hooks will catch any security misconfigurations or deprecated resources before code is committed because, those bring about errors or security leaks when the project repo is committed.

Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include Terraform, GitHub Actions, Amazon S3, Dynamo DB, Amazon RDS, Amazon VPC and AWS EC2 . Key concepts I learnt include how to:

- Build a modular Terraform codebase provisioning a complete AWS stack with networking, compute, storage, and IAM modules.
- Deploy infrastructure changes through a GitOps pipeline using GitHub Actions with passwordless OIDC authentication to AWS.

Time and challenges

This project took me approximately 7hrs(including building the Terraform modular codebase and troubleshooting some few CI/CD pipeline errors with GitHub actions). The most challenging part was establishing a full pipeline that easily deploys infrastructure on AWS through Terraform.

What's next

I did this project today to learn how Production teams use Terraform to deploy and manage infrastructure on Cloud Providers such as AWS. Another skill I want to learn is how to utilize workspaces in Terraform within this project, so deployment in different environments becomes way easier with just a few lines of code.